



Pontifícia Universidade Católica de Minas Gerais
Departamento de Ciência da Computação
Disciplina: Compiladores - Prova 3 (25 pts)
Professor: Pedro Ramos

Estudante: _____

1. Análise de Fluxo (9pts)

```
1 g(y) {  
2 B1 :  
3     i := 0  
4     a := 5 * y  
5     if (a > 10) goto B3  
6 B2 :  
7     i := i + 1  
8     x := a + 1  
9     if (i < 10) goto B2  
10 B3 :  
11     x := a + 1  
12     return x  
13 }
```

- Desenhe o CFG (Grafo de Fluxo de Controle) do programa acima no nível dos **blocos básicos**. (1pt)
- Redesenhe o CFG no nível das **instruções**. (1pt)
- Faça a análise de vida das variáveis e indique os conjuntos IN e OUT de cada **instrução** do programa. (2pts)
- Utilizando o conceito de variáveis vivas, desenhe o Grafo de Interferência de Registradores (RIG) do programa. (2pts)
- Uma otimização conhecida é a **Movimentação de Código Invariante**. Qual ou quais expressões no programa são invariantes e podem ser movimentadas no código acima? Indique (i) a ou as expressões invariantes; e (ii) para qual bloco básico cada expressão invariante identificada pode ser seguramente movida, e **por qual motivo** o compilador realizaria tal movimentação. (3pts)

2. **Otimização de Código I (8pts)** Para o programa abaixo, considere as duas otimizações a seguir:

O_1 : Eliminação de Código Morto

O_2 : Propagação de Constantes

```
1 h() {
2 B1:
3   x := 5
4   a := 10
5   i := stdin()
6   goto B2
7
8 B2:
9   t1 := i % 2
10  t2 := t1 == 0
11  if !t2 goto B4
12
13 B3:
14  if x == 5 goto B5 else goto B6
15
16 B4:
17  b := a + 1
18  y := 20
19  goto B7
20
21 B5:
22  y := 1
23  z := x + y
24  goto B7
25
26 B6:
27  c := 99
28  goto B7
29
30 B7:
31  return z
32 }
```

(a) Responda: existe uma ordem ótima para a aplicação de O_1 e O_2 neste programa específico? Justifique. (2pts)

(b) Aplique as duas otimizações na melhor ordem possível e mostre o resultado final. (4pts)

(c) Cite pelo menos mais uma otimização além de O_1 e O_2 que poderia ser aplicada ao programa acima e que resultaria em um programa menor ou mais eficiente que o atual. (2pts)

3. Otimização de Código II (4pts)

Para a lista de otimizações a seguir, preencha a tabela de descrição de otimizações abaixo com o número da otimização correspondente.

- 1. Redução de Força
- 2. Inlining de Funções
- 3. Propagação de Constantes
- 4. Eliminação de Código Morto
- 5. Eliminação de Subexpressões Comuns
- 6. Duplicação de Cauda
- 7. Eliminação de Recursão de Cauda
- 8. Mover Invariante de Loop
- 9. Análise de Ponteiros
- 10. Vetorização
- 11. Desenrolamento de Loop
- 12. Agendamento de Instruções (*Instruction Scheduling*)
- 13. Alocação de Registradores

Otimização (Nº)	Descrição
	Remove instruções que definem valores em variáveis que nunca são lidas ou usadas posteriormente no programa, ou remove blocos inatingíveis.
	Converte código sequencial para aproveitar instruções de processador que operam em múltiplos dados simultaneamente (SIMD).
	Mapeia variáveis de programa (temporárias) para os registradores físicos do hardware, minimizando a necessidade de <i>spilling</i> para a memória.
	Duplica um bloco de junção para todos os seus blocos predecessores.
	Substitui operações custosas por operações mais baratas.
	Determina se diferentes ponteiros ou nomes de variáveis referenciam a mesma localização na memória.
	Substitui o uso de uma expressão que já foi calculada e salva, evitando o recálculo redundante.
	Reorganiza a ordem das instruções dentro de um mesmo bloco básico para minimizar latência de pipeline.
	Transfere cálculos que produzem o mesmo resultado em cada iteração de um laço para fora do corpo do laço.
	Copia o corpo de uma função chamada diretamente no ponto de invocação.
	Executa múltiplas iterações de um laço em uma única iteração, reduzindo a sobrecarga de controle do loop.
	Transforma chamadas de função no final de uma função em um salto (<i>jump</i>) simples para o início da mesma, reutilizando o registro de ativação.
	Substitui o uso de variáveis por seus valores constantes conhecidos em tempo de compilação, o que pode habilitar a eliminação de código.

4. Gerenciamento de Memória (4pts)

(a) O Coletor de Lixo por **Contagem de Referências**, usado pelo Python, tem a vantagem de ser incremental, mas a desvantagem de impor uma sobrecarga significativa em cada operação de atribuição. Essa sobrecarga deve ser realizada de forma atômica? Justifique. (2pts)

(b) A técnica de reorganização do layout do *heap* utilizada pelo Coletor de Lixo **Stop & Copy** melhora a localidade de cache dos objetos vivos, mesmo que o compilador não otimize o layout interno dos objetos. Por quê isso acontece? Exemplifique com desenho ou diagrama e **justifique sua resposta**. (2pts)